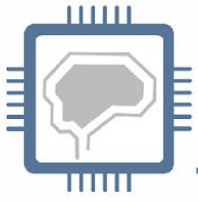


***Inteligência Artificial
e Computacional***

Disciplina: Linguagens de Programação para Inteligência Artificial

Aula 2: Comandos básicos da linguagem Python - parte I

Profa.: Sabrina de A. Silveira

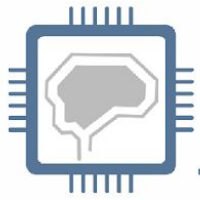


Atribuição

- O comando de atribuição associa um valor a uma variável.
- O resultado à direita da expressão é atribuído ao operando à esquerda da expressão

$$x = 5 + 3$$

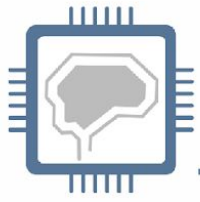
A variável x conterá o valor 8



Atribuição

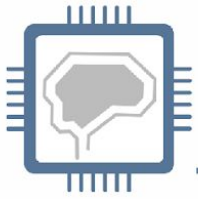
Apresentamos os principais operadores de atribuição, com exceção dos que trabalham bit a bit.

Operador	Descrição	Exemplo
=	Atribui o valor da expressão ao lado direito do operador ao operando da esquerda	$x = y + z$
+=	Soma os dois operandos e atribui ao da esquerda	$a += b$
-=	Subtrai os dois operandos e atribui ao da esquerda	$a -= b$
*=	Multiplica os dois operandos e atribui ao da esquerda	$a *= b$
/=	Divide os dois operandos e atribui ao da esquerda	$a /= b$
%=	Calcula $a \bmod b$ e atribui ao da esquerda	$a \% = b$
//=	Calcula $a // b$ e atribui ao da esquerda	$a //= b$
**=	Calcula $a ** b$ e atribui ao da esquerda	$a ** = b$



Entrada e saída padrão

- O dispositivo de entrada padrão é o **teclado**.
- A função **input** permite receber algum texto digitado pelo usuário no teclado.
- Essa função pausa a execução do programa e aguarda que o usuário digite algum texto e pressione a tecla **Enter**.
- A entrada recebida do usuário pode ser atribuída a uma variável para que possa ser utilizada no programa



Entrada e saída padrão

➔ Pequeno programa que usa a função input:

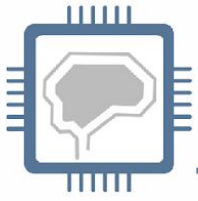
```
message = input("Diga alguma coisa e eu vou repetir: ")  
print(message)
```

Ao ser executado, o programa aguarda que o usuário digite:

```
Diga alguma coisa e eu vou repetir: Olá pessoal!
```

Programa imprime o que o usuário digitou:

```
Diga alguma coisa e eu vou repetir: Olá pessoal!  
Olá pessoal!
```



Entrada e saída padrão

Pequeno programa que usa a função input:

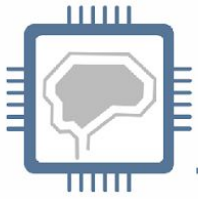
```
message = input("Diga alguma coisa e eu vou repetir: ")  
print(message)
```

➔ Ao ser executado, o programa aguarda que o usuário digite:

```
Diga alguma coisa e eu vou repetir: Olá pessoal!
```

Programa imprime o que o usuário digitou:

```
Diga alguma coisa e eu vou repetir: Olá pessoal!  
Olá pessoal!
```

Entrada e saída padrão

Pequeno programa que usa a função input:

```
message = input("Diga alguma coisa e eu vou repetir: ")  
print(message)
```

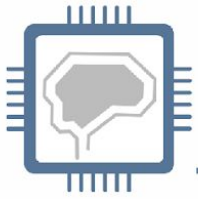
Ao ser executado, o programa aguarda que o usuário digite:

```
Diga alguma coisa e eu vou repetir: Olá pessoal!
```



Programa imprime o que o usuário digitou:

```
Diga alguma coisa e eu vou repetir: Olá pessoal!  
Olá pessoal!
```

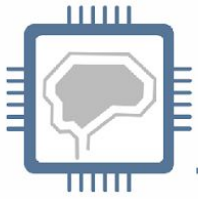


Entrada e saída padrão

- Quando a função `input` é utilizada, o interpretador Python assume que a entrada fornecida pelo usuário é do tipo **string**;
- Se a entrada for um **valor numérico** inteiro ou de ponto flutuante, isso deve ser **sinalizado explicitamente**.

```
idade = int(input('Qual sua idade? '))
altura = float(input('Sua altura (m)? '))
print('Idade:', idade)
print('Altura:', altura, 'm')
```

```
Qual sua idade? 12
Sua altura (m)? 1.56
Idade: 12
Altura: 1.56 m
```

Entrada e saída padrão

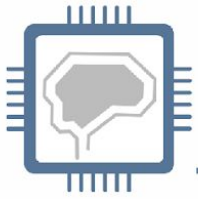
- Quando a função `input` é utilizada, o interpretador Python assume que a entrada fornecida pelo usuário é do tipo **string**;
- Se a entrada for um **valor numérico** inteiro ou de ponto flutuante, isso deve ser **sinalizado explicitamente**.

Entradas do tipo int e float

```
idade = int(input('Qual sua idade? '))
altura = float(input('Sua altura (m)? '))
print('Idade:', idade)
print('Altura:', altura, 'm')
```

Resultado da execução

```
Qual sua idade? 12
Sua altura (m)? 1.56
Idade: 12
Altura: 1.56 m
```



Entrada e saída padrão

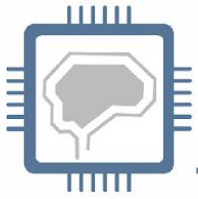
- Quando a função `input` é utilizada, o interpretador Python assume que a entrada fornecida pelo usuário é do tipo **string**;
- Se a entrada for um **valor numérico** inteiro ou de ponto flutuante, isso deve ser **sinalizado explicitamente**.

Entradas do tipo int e float

```
idade = int(input('Qual sua idade? '))  
altura = float(input('Sua altura (m)? '))  
print('Idade:', idade)  
print('Altura:', altura, 'm')
```

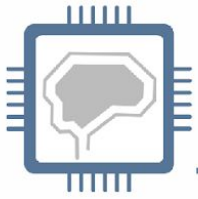
Resultado da execução

```
Qual sua idade? 12  
Sua altura (m)? 1.56  
Idade: 12  
Altura: 1.56 m
```



Entrada e saída padrão

- Com a função `print` podemos imprimir mensagens na tela, que é a saída padrão.
- A mensagem pode ser um string ou outro objeto que será convertido em string para ser impresso.
- Alguns parâmetros importantes dessa função são:
 - **objeto(s)**: um ou mais objetos que serão impressos;
 - **sep='separator'**: define qual será o separador utilizado para imprimir os objetos;
 - **end='end'**: terminador impresso ao final da impressão



Entrada e saída padrão

Diferentes tipos de separador podem ser usados. O mesmo vale para o último caracter impresso na tela.

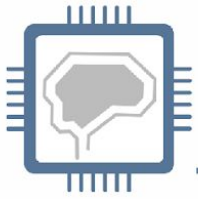
```
print("banana")  
print("banana", "maçã", "abacaxi")  
print("banana", "maçã", "abacaxi", sep=":")  
print("banana", "maçã", "abacaxi", sep=":", end="-")
```

banana

banana maçã abacaxi

banana:maçã:abacaxi

banana:maçã:abacaxi-

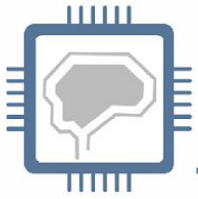


Entrada e saída padrão

Diferentes tipos de separador podem ser usados. O mesmo vale para o último caracter impresso na tela.

Imprimindo 1 objeto → `print("banana")`
`print("banana", "maçã", "abacaxi")`
`print("banana", "maçã", "abacaxi", sep=":")`
`print("banana", "maçã", "abacaxi", sep=":", end="-")`

→ `banana`
`banana maçã abacaxi`
`banana:maçã:abacaxi`
`banana:maçã:abacaxi-`



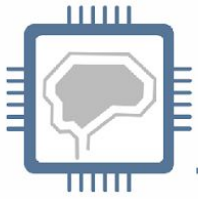
Entrada e saída padrão

Diferentes tipos de separador podem ser usados. O mesmo vale para o último caracter impresso na tela.

Imprimindo 3 objetos →

```
print("banana")  
print("banana", "maçã", "abacaxi")  
print("banana", "maçã", "abacaxi", sep=":")  
print("banana", "maçã", "abacaxi", sep=":", end="-")
```

```
banana  
→ banana maçã abacaxi  
banana:maçã:abacaxi  
banana:maçã:abacaxi-
```

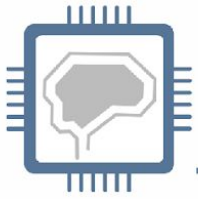
Entrada e saída padrão

Diferentes tipos de separador podem ser usados. O mesmo vale para o último caracter impresso na tela.

Alterando separador →

```
print("banana")  
print("banana", "maçã", "abacaxi")  
print("banana", "maçã", "abacaxi", sep=":")  
print("banana", "maçã", "abacaxi", sep=":", end="-")
```

```
banana  
banana maçã abacaxi  
→ banana:maçã:abacaxi  
banana:maçã:abacaxi-
```



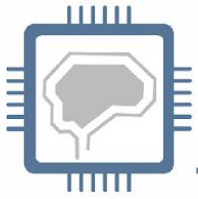
Entrada e saída padrão

Diferentes tipos de separador podem ser usados. O mesmo vale para o último caracter impresso na tela.

Alterando separador e
último caracter impresso

```
print("banana")  
print("banana", "maçã", "abacaxi")  
print("banana", "maçã", "abacaxi", sep=":")  
→ print("banana", "maçã", "abacaxi", sep=":", end="-")
```

```
banana  
banana maçã abacaxi  
banana:maçã:abacaxi  
→ banana:maçã:abacaxi-
```



Estruturas condicionais

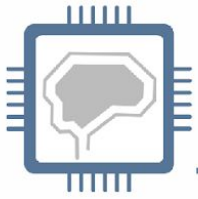
Nos permitem testar uma condição e direcionar o fluxo de execução do programa para um ponto específico dentro do código dependendo se a condição é falsa ou verdadeira.

Se a condição é verdadeira, o comando `print` é executado.

```
age = 19
if age >= 18:
    print("O voto é obrigatório!")
```

Saída do programa, mostrando que o comando `print` foi executado.

O voto é obrigatório!



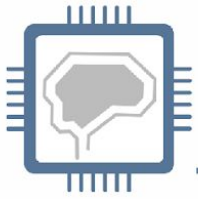
Estruturas condicionais

É comum precisar executar determinado conjunto de comandos somente se a condição for verdadeira e outro conjunto de comandos caso a condição testada seja falsa.

Se a condição é verdadeira,
entra aqui

```
age = 17
if age >= 18:
    print("Você tem a idade mínima para dirigir.")
else:
    print("Você ainda não tem idade mínima para dirigir.")
```

Você ainda não tem idade mínima para dirigir.



Estruturas condicionais

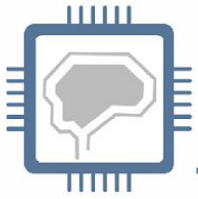
É comum precisar executar determinado conjunto de comandos somente se a condição for verdadeira e outro conjunto de comandos caso a condição testada seja falsa.

Se a condição é verdadeira,
entra aqui

Caso contrário, entra aqui

```
age = 17
if age >= 18:
    print("Você tem a idade mínima para dirigir.")
else:
    → print("Você ainda não tem idade mínima para dirigir.")
```

Você ainda não tem idade mínima para dirigir.



Estruturas condicionais

É comum precisar executar determinado conjunto de comandos somente se a condição for verdadeira e outro conjunto de comandos caso a condição testada seja falsa.

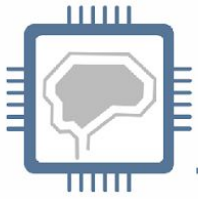
Se a condição é verdadeira,
entra aqui

Caso contrário, entra aqui

```
age = 17
if age >= 18:
    print("Você tem a idade mínima para dirigir.")
else:
    print("Você ainda não tem idade mínima para dirigir.")
```

Como a condição no
programa é falsa...

➡ Você ainda não tem idade mínima para dirigir.



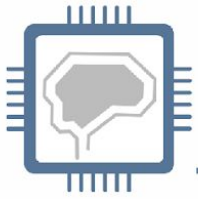
Estruturas condicionais

Há casos em que existem mais de duas possíveis situações e os comandos if-elif-else podem ser utilizados para avaliá-las.

Se idade < 16 entra aqui

```
age = 19
if age < 16:
    print("Você ainda não pode votar.")
elif age < 18:
    print("Você já pode votar.")
elif age < 70:
    print("O voto é obrigatório.")
else:
    print("Você pode votar.")
```

O voto é obrigatório.



Estruturas condicionais

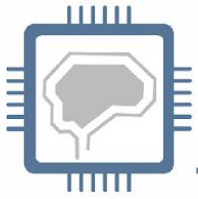
Há casos em que existem mais de duas possíveis situações e os comandos if-elif-else podem ser utilizados para avaliá-las.

Se idade < 16 entra aqui

Caso contrário, se idade < 18 entra aqui

```
age = 19
if age < 16:
    print("Você ainda não pode votar.")
elif age < 18:
    print("Você já pode votar.")
elif age < 70:
    print("O voto é obrigatório.")
else:
    print("Você pode votar.")
```

O voto é obrigatório.



Estruturas condicionais

Há casos em que existem mais de duas possíveis situações e os comandos if-elif-else podem ser utilizados para avaliá-las.

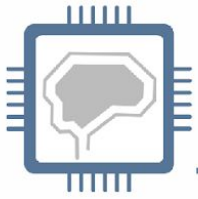
Se idade < 16 entra aqui

Caso contrário, se idade < 18 entra aqui

Caso contrário, se idade < 70 entra aqui

```
age = 19
if age < 16:
    print("Você ainda não pode votar.")
elif age < 18:
    print("Você já pode votar.")
elif age < 70:
    print("0 voto é obrigatório.")
else:
    print("Você pode votar.")
```

0 voto é obrigatório.



Estruturas condicionais

Há casos em que existem mais de duas possíveis situações e os comandos if-elif-else podem ser utilizados para avaliá-las.

Se idade < 16 entra aqui

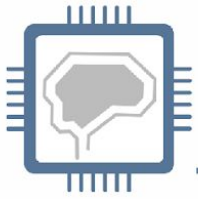
Caso contrário, se idade < 18 entra aqui

Caso contrário, se idade < 70 entra aqui

Se nenhuma das anteriores, entra aqui

```
age = 19
if age < 16:
    print("Você ainda não pode votar.")
elif age < 18:
    print("Você já pode votar.")
elif age < 70:
    print("O voto é obrigatório.")
else:
    → print("Você pode votar.")
```

O voto é obrigatório.



Estruturas condicionais

Há casos em que existem mais de duas possíveis situações e os comandos if-elif-else podem ser utilizados para avaliá-las.

Se idade < 16 entra aqui

Caso contrário, se idade < 18 entra aqui

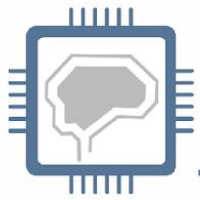
Caso contrário, se idade < 70 entra aqui

Se nenhuma das anteriores, entra aqui

```
age = 19
if age < 16:
    print("Você ainda não pode votar.")
elif age < 18:
    print("Você já pode votar.")
elif age < 70:
    print("O voto é obrigatório.")
else:
    print("Você pode votar.")
```

Como a idade é 19...

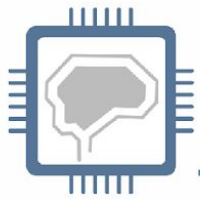
→ O voto é obrigatório.



Operadores lógicos

Os operadores lógicos permitem executar as operações lógicas AND, OR e NOT. São frequentemente utilizados com comandos condicionais.

Operador	Descrição	Exemplo
and	AND: True se os 2 operandos são verdadeiros	x and y
or	OR: True se qualquer dos 2 operandos for verdadeiro	x or y
not	NOT: True se o operando for falso	not x



Operadores lógicos

O operador **and** resultará em verdadeiro somente se todos os operandos envolvidos forem verdadeiros.

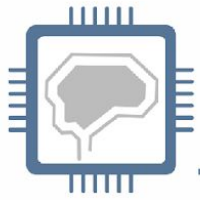
```
>>> x = True
>>> y = True
>>> x and y
True_
```

```
>>> x = True
>>> y = False
>>> x and y
False
```

```
>>> x = False
>>> y = True
>>> x and y
False
```

```
>>> x = False
>>> y = False
>>> x and y
False_
```

x	y	x and y
True	True	True
True	False	False
False	True	False
False	False	False



Operadores lógicos

O operador **and** resultará em verdadeiro somente se todos os operandos envolvidos forem verdadeiros.

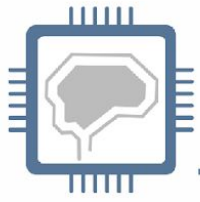
```
>>> x = True
>>> y = True
→ >>> x and y
True_
```

```
>>> x = True
>>> y = False
>>> x and y
False
```

```
>>> x = False
>>> y = True
>>> x and y
False
```

```
>>> x = False
>>> y = False
>>> x and y
False_
```

x	y	x and y
True	True	True
True	False	False
False	True	False
False	False	False



Operadores lógicos

O operador **and** resultará em verdadeiro somente se todos os operandos envolvidos forem verdadeiros.

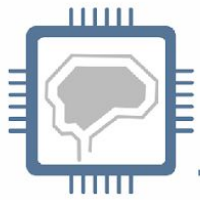
```
>>> x = True
>>> y = True
>>> x and y
True_
```

```
>>> x = True
>>> y = False
→ >>> x and y
False
```

```
>>> x = False
>>> y = True
>>> x and y
False
```

```
>>> x = False
>>> y = False
>>> x and y
False_
```

x	y	x and y
True	True	True
True	False	False
False	True	False
False	False	False



Operadores lógicos

O operador **and** resultará em verdadeiro somente se todos os operandos envolvidos forem verdadeiros.

```
>>> x = True
>>> y = True
>>> x and y
True_
```

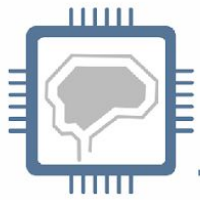
```
>>> x = True
>>> y = False
>>> x and y
False
```



```
>>> x = False
>>> y = True
>>> x and y
False
```

```
>>> x = False
>>> y = False
>>> x and y
False_
```

x	y	x and y
True	True	True
True	False	False
False	True	False
False	False	False



Operadores lógicos

O operador **and** resultará em verdadeiro somente se todos os operandos envolvidos forem verdadeiros.

```
>>> x = True
>>> y = True
>>> x and y
True_
```

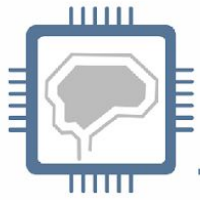
```
>>> x = True
>>> y = False
>>> x and y
False
```

```
>>> x = False
>>> y = True
>>> x and y
False
```

```
>>> x = False
>>> y = False
>>> x and y
False_
```



x	y	x and y
True	True	True
True	False	False
False	True	False
False	False	False



Operadores lógicos

O operador **or** resultará em verdadeiro se qualquer um dos operandos envolvidos for verdadeiro.

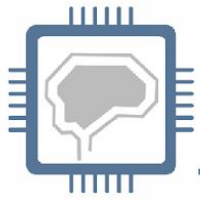
```
>>> x = True
>>> y = True
>>> x or y
True
```

```
>>> x = True
>>> y = False
>>> x or y
True
```

```
>>> x = False
>>> y = True
>>> x or y
True
```

```
>>> x = False
>>> y = False
>>> x or y
False
```

x	y	x or y
True	True	True
True	False	True
False	True	True
False	False	False



Operadores lógicos

O operador **or** resultará em verdadeiro se qualquer um dos operandos envolvidos for verdadeiro.

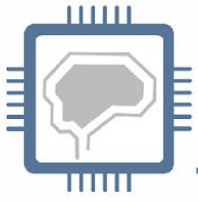
```
>>> x = True
>>> y = True
→ >>> x or y
True
```

```
>>> x = True
>>> y = False
>>> x or y
True
```

```
>>> x = False
>>> y = True
>>> x or y
True
```

```
>>> x = False
>>> y = False
>>> x or y
False
```

x	y	x or y
True	True	True
True	False	True
False	True	True
False	False	False



Operadores lógicos

O operador **or** resultará em verdadeiro se qualquer um dos operandos envolvidos for verdadeiro.

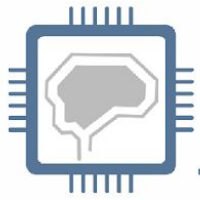
```
>>> x = True
>>> y = True
>>> x or y
True
```

```
>>> x = False
>>> y = True
>>> x or y
True
```

```
>>> x = True
>>> y = False
→ >>> x or y
True
```

```
>>> x = False
>>> y = False
>>> x or y
False
```

x	y	x or y
True	True	True
True	False	True
False	True	True
False	False	False



Operadores lógicos

O operador **or** resultará em verdadeiro se qualquer um dos operandos envolvidos for verdadeiro.

```
>>> x = True
>>> y = True
>>> x or y
True
```

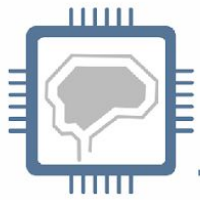


```
>>> x = False
>>> y = True
>>> x or y
True
```

```
>>> x = True
>>> y = False
>>> x or y
True
```

```
>>> x = False
>>> y = False
>>> x or y
False
```

x	y	x or y
True	True	True
True	False	True
False	True	True
False	False	False



Operadores lógicos

O operador **or** resultará em verdadeiro se qualquer um dos operandos envolvidos for verdadeiro.

```
>>> x = True
>>> y = True
>>> x or y
True
```

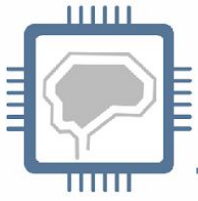
```
>>> x = True
>>> y = False
>>> x or y
True
```

```
>>> x = False
>>> y = True
>>> x or y
True
```

```
>>> x = False
>>> y = False
>>> x or y
False
```



x	y	x or y
True	True	True
True	False	True
False	True	True
False	False	False



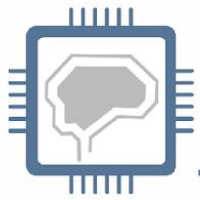
Operadores lógicos

O operador **not** é unário, o que significa que ele atua sobre um operando apenas. Ele inverte o valor do operando.

```
>>> x = True
→ >>> not x
False
```

```
>>> x = False
>>> not x
True
```

x	not x
True	False
False	True



Operadores lógicos

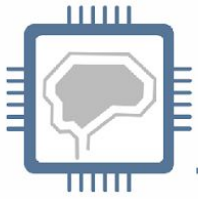
O operador **not** é unário, o que significa que ele atua sobre um operando apenas. Ele inverte o valor do operando.

```
>>> x = True
>>> not x
False
```



```
>>> x = False
>>> not x
True
```

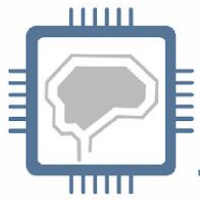
x	not x
True	False
False	True



Precedência de operadores

Apresentamos novamente a precedência de operadores em Python, agora incluindo os operadores relacionais e lógicos, além dos aritméticos.

Precedência	Operadores e comandos
1	- (unário)
2	**
3	*, /, // %
4	+, -
5	<, <=, >, >=, == !=
6	not
7	and
8	or
9	= (atribuição)



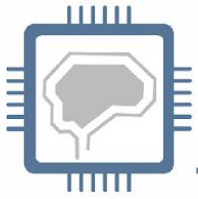
Operadores lógicos

Os comandos condicionais podem ser usados em conjunto com operadores relacionais e lógicos. Neste exemplo, a nota do aluno é avaliada e ele pode ser considerado aprovado, reprovado ou ter direito a fazer prova final.

```
nota = float(input('Nota final: '))
if nota < 0 or nota > 100:
    print('Nota informada é inválida.')
elif nota >= 60:
    print('Aluno aprovado.')
elif nota >= 40:
    print('Aluno tem direito a fazer prova final.')
else:
    print('Aluno reprovado.')
```

Nota final: 58

Aluno tem direito a fazer prova final.



Operadores lógicos

Os comandos condicionais podem ser usados em conjunto com operadores relacionais e lógicos. Neste exemplo, nota do aluno é avaliada ele pode ser considerado aprovado, reprovado ou ter direito a fazer prova final.

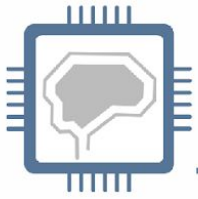
O programa pede ao usuário que digite a nota ...



```
nota = float(input('Nota final: '))
if nota < 0 or nota > 100:
    print('Nota informada é inválida.')
elif nota >= 60:
    print('Aluno aprovado.')
elif nota >= 40:
    print('Aluno tem direito a fazer prova final.')
else:
    print('Aluno reprovado.')
```

Nota final: 58

Aluno tem direito a fazer prova final.



Operadores lógicos

Os comandos condicionais podem ser usados em conjunto com operadores relacionais e lógicos. Neste exemplo, nota do aluno é avaliada ele pode ser considerado aprovado, reprovado ou ter direito a fazer prova final.

O programa pede ao usuário que digite a nota ...

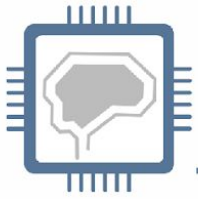
Se nota não está entre 0 e 100



```
nota = float(input('Nota final: '))
if nota < 0 or nota > 100:
    print('Nota informada é inválida.')
elif nota >= 60:
    print('Aluno aprovado.')
elif nota >= 40:
    print('Aluno tem direito a fazer prova final.')
else:
    print('Aluno reprovado.')
```

Nota final: 58

Aluno tem direito a fazer prova final.



Operadores lógicos

Os comandos condicionais podem ser usados em conjunto com operadores relacionais e lógicos. Neste exemplo, nota do aluno é avaliada ele pode ser considerado aprovado, reprovado ou ter direito a fazer prova final.

O programa pede ao usuário que digite a nota ...

Se nota não está entre 0 e 100

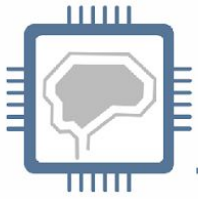
Caso contrário, se nota ≥ 60



```
nota = float(input('Nota final: '))
if nota < 0 or nota > 100:
    print('Nota informada é inválida.')
elif nota >= 60:
    print('Aluno aprovado.')
elif nota >= 40:
    print('Aluno tem direito a fazer prova final.')
else:
    print('Aluno reprovado.')
```

Nota final: 58

Aluno tem direito a fazer prova final.



Operadores lógicos

Os comandos condicionais podem ser usados em conjunto com operadores relacionais e lógicos. Neste exemplo, nota do aluno é avaliada ele pode ser considerado aprovado, reprovado ou ter direito a fazer prova final.

O programa pede ao usuário que digite a nota ...

Se nota não está entre 0 e 100

Caso contrário, se nota ≥ 60

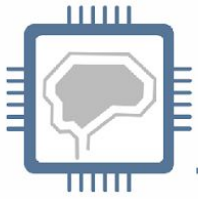
Caso contrário, se nota ≥ 40



```
nota = float(input('Nota final: '))
if nota < 0 or nota > 100:
    print('Nota informada é inválida.')
elif nota >= 60:
    print('Aluno aprovado.')
elif nota >= 40:
    print('Aluno tem direito a fazer prova final.')
else:
    print('Aluno reprovado.')
```

Nota final: 58

Aluno tem direito a fazer prova final.



Operadores lógicos

Os comandos condicionais podem ser usados em conjunto com operadores relacionais e lógicos. Neste exemplo, nota do aluno é avaliada ele pode ser considerado aprovado, reprovado ou ter direito a fazer prova final.

O programa pede ao usuário que digite a nota ...

Se nota não está entre 0 e 100

Caso contrário, se nota ≥ 60

Caso contrário, se nota ≥ 40

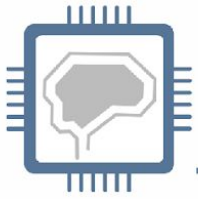
Se nenhuma das anteriores

```
nota = float(input('Nota final: '))
if nota < 0 or nota > 100:
    print('Nota informada é inválida.')
elif nota >= 60:
    print('Aluno aprovado.')
elif nota >= 40:
    print('Aluno tem direito a fazer prova final.')
else:
    print('Aluno reprovado.')
```



Nota final: 58

Aluno tem direito a fazer prova final.



Operadores lógicos

Os comandos condicionais podem ser usados em conjunto com operadores relacionais e lógicos. Neste exemplo, nota do aluno é avaliada ele pode ser considerado aprovado, reprovado ou ter direito a fazer prova final.

O programa pede ao usuário que digite a nota ...

Se nota não está entre 0 e 100

Caso contrário, se nota ≥ 60

Caso contrário, se nota ≥ 40

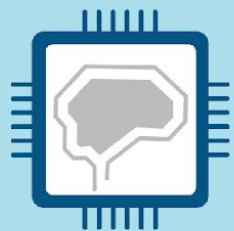
Se nenhuma das anteriores

Para a nota informada, 58, essa é a saída

```
nota = float(input('Nota final: '))
if nota < 0 or nota > 100:
    print('Nota informada é inválida.')
elif nota >= 60:
    print('Aluno aprovado.')
elif nota >= 40:
    print('Aluno tem direito a fazer prova final.')
else:
    print('Aluno reprovado.')
```



```
Nota final: 58
Aluno tem direito a fazer prova final.
```



Inteligência Artificial e Computacional

Profa.: Sabrina de A. Silveira